

Problem 1:

A hospital records patient visits throughout the day. Each visit contains the patient's name, the severity level of the illness, and the treatment time in minutes.

Each record consists of:

patient_name severity treatment_time

where:

- **patient_name** is a string representing the patient's name.
- **severity** is an integer (1–10).
- **treatment_time** is an integer representing the treatment duration (in minutes).

Tasks

1. Store every patient visit as a tuple (**patient_name**, **severity**, **treatment_time**) inside a list.
2. Build a dictionary where each patient stores:
 - **total_treatment_time** = Sum of all treatment times.
 - **total_severity_score** = Sum of all severity values.
3. Print the summary for every patient in **alphabetical order**.
4. Print the patient having the **highest total severity score**.
 - If multiple patients have the same highest severity score, print the **lexicographically smaller** patient name.

Input Format

- The first line contains an integer **N**, the number of patient visits.
- The next **N** lines each contain:

patient_name severity treatment_time

Output Format

For each patient, print:

patient_name total_treatment_time total_severity_score

(sorted alphabetically)

Finally print

TOP patient_name total_severity_score

Sample Input

```
7
Alice 5 30
Bob 3 20
Alice 4 25
Charlie 7 15
Bob 6 40
Charlie 2 30
Alice 3 20
```

Sample Output

```
Alice 75 12
Bob 60 9
Charlie 45 9
TOP Alice 12
```

Problem 2:

A company has **S salespersons** selling **M different products**. Their monthly sales are stored in a matrix **A** of size **S × M**.

You are also given a vector **target_sales** containing the monthly target for each product.

Your task is to analyze the performance of the salespersons.

Tasks

1. Compute the **achievement percentage matrix** using:

$$P[i, j] = 100 * \frac{A[i, j]}{target_sales[j]}$$

2. For every salesperson compute:
 - Average achievement percentage.
 - Best-performing product (highest percentage).
3. For every product compute:
 - Average achievement percentage.
 - Top-performing salesperson.
4. Print the IDs of the **Top K salespersons** based on average achievement.
5. Convert every percentage into grades:

Percentage	Grade
≥ 90	Excellent
[75, 90)	Good
[60, 75)	Average
< 60	Poor

Count the total number of each grade.

Input Format

- Number of salespersons (**S**)
- Number of products (**M**)
- Sales matrix (**S** × **M**)
- Target sales vector
- Integer **K**

Output Format

Print:

- Percentage matrix
- Salesperson summary
- Product summary
- Top K salesperson IDs
- Grade counts

Sample Input

```
3
3
80 70 90
60 85 75
95 90 80
100 100 100
2
```

Sample Output

Percentage Matrix

80.00 70.00 90.00

60.00 85.00 75.00

95.00 90.00 80.00

Salesperson Summary

Salesperson 0 : Average = 80.00 Best Product = 2

Salesperson 1 : Average = 73.33 Best Product = 1

Salesperson 2 : Average = 88.33 Best Product = 0

Product Summary

Product 0 : Average = 78.33 Top Salesperson = 2

Product 1 : Average = 81.67 Top Salesperson = 2

Product 2 : Average = 81.67 Top Salesperson = 0

Top 2 Salespersons

2

0

Grade Count

Excellent : 3

Good : 4

Average : 2

Poor : 0

Problem 3:

Create an **Employee** class to manage employee salary information.

Each employee has

- Name
- Employee ID
- List of monthly salaries

Class Attribute

Company Name

Instance Attributes

- name
- employee_id
- salaries

Methods

Implement the following methods:

- `__init__()`
- `add_salary(self, salary)`
- `average_salary(self)`
- `highest_salary(self)`
- `annual_income(self)`
- `__str__(self)`

Tasks

1. Create **5 Employee** objects.
2. Each employee should have salary records for **12 months**.
3. Display:

- Employee Information
- Average Salary
- Highest Salary
- Annual Income

Sample Output

Employee Name : Alice

Employee ID : E101

Average Salary : 46500.50

Highest Salary : 52000

Annual Income : 558006

Problem 4:

A company operates three branches: **North**, **South**, and **Central**. The monthly profit (in thousand dollars) of each branch during the year **2025** is stored in three NumPy arrays.

Use the following data:

- months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
- north = np.array([120, 135, 128, 142, 150, 158, 162, 170, 168, 180, 185, 195])
- south = np.array([110, 118, 125, 130, 138, 145, 150, 155, 160, 168, 172, 180])
- central = np.array([100, 108, 115, 120, 130, 140, 148, 152, 158, 165, 170, 175])

Using **Matplotlib**, create a single figure consisting of **four subplots** arranged in a **2 × 2** grid to visualize the company's branch performance.

Requirements

Plot 1 (Top Left): Monthly Profit Line Plot

Plot the monthly profit of all three branches on the same graph using the following styles:

- **North:** Blue solid line
- **South:** Green dashed line
- **Central:** Red dotted line

Include:

- Title: **Monthly Branch Profit (2025)**
- X-axis label: **Month**
- Y-axis label: **Profit (in thousand dollars)**
- Legend
- Grid

Plot 2 (Top Right): December Profit Comparison

Create a **bar chart** showing the **December profit** of the three branches.

Requirements:

- Use different colors for each branch.

- Title: **December Profit Comparison**
- X-axis label: **Branch**
- Y-axis label: **Profit (in thousand dollars)**
- Enable the grid on the **y-axis** only.

Plot 3 (Bottom Left): North Branch Profit Distribution

Create a **scatter plot** showing the monthly profits of the **North** branch.

Requirements:

- Blue markers
- Marker size = **60**
- Title: **North Branch Profit Distribution**
- X-axis label: **Month**
- Y-axis label: **Profit (in thousand dollars)**
- Grid

Plot 4 (Bottom Right): Quarterly Branch Profit

Compute the quarterly profit for each branch using the following definitions:

- **Q1:** January – March
- **Q2:** April – June
- **Q3:** July – September
- **Q4:** October – December

Create a **stacked bar chart** showing the quarterly profits of the three branches.

Include:

- Title: **Quarterly Branch Profit**
- X-axis label: **Quarter**
- Y-axis label: **Profit (in thousand dollars)**
- Legend
- Grid on the y-axis

Figure Settings

- Create a figure of size **(14, 10)**.

- Add the main title:
Company Branch Performance Analysis (2025)
- Arrange the subplots neatly using `plt.tight_layout()`.

